

Introduction to Functions

This lecture will consist of explaining what a function is in R and how to create one. Functions will be one of our main building blocks when we construct larger and larger amounts of code to solve problems.

So what is a function?

Formally, a function is a useful device that groups together a set of statements so they can be run more than once. They can also let us specify parameters that can serve as inputs to the functions.

On a more fundamental level, functions allow us to not have to repeatedly write the same code again and again. If you remember back to the lessons on strings and lists, remember that we used a function `length()` to get the length of a string. Since checking the length of a sequence is a common task you would want to write a function that can do this repeatedly at command. Functions will be one of most basic levels of reusing code in R, and it will also allow us to start thinking about program design.

We already have seen built-in functions and we can use the **help** function to discover the *arguments* that the functions take in.

In [1]:

```
help(sum)
```

Out[1]:

sum {base}	R Documentation
------------	-----------------

Sum of Vector Elements

Description

`sum` returns the sum of all the values present in its arguments.

Usage

```
sum(..., na.rm = FALSE)
```

Arguments

...	numeric or complex or logical vectors.
na.rm	logical. Should missing values (including NaN) be removed?

Details

This is a generic function: methods can be defined for it directly or via the `Summary` group generic. For this to work properly, the arguments ... should be unnamed, and dispatch is on the first argument.

If `na.rm` is `FALSE` an `NA` or `NaN` value in any of the arguments will cause a value of `NA` or `NaN` to be returned, otherwise `NA` and `NaN` values are ignored.

Logical true values are regarded as one, false values as zero. For historical reasons, `NULL` is accepted and treated as if it were `integer(0)`.

Loss of accuracy can occur when summing values of different signs: this can even occur for sufficiently long integer inputs if the partial sums would cause integer overflow. Where possible extended-precision accumulators are used, but this is platform-dependent.

Value

The sum. If all of ... are of type integer or logical, then the sum is integer, and in that case the result will be `NA` (with a warning) if integer overflow occurs. Otherwise it is a length-one numeric or complex vector.

NB: the sum of an empty set is zero, by definition.

S4 methods

This is part of the S4 Summary group generic. Methods for it must use the signature `x, ..., na.rm`.

'plotmath' for the use of `sum` in plot annotation.

References

Becker, R. A., Chambers, J. M. and Wilks, A. R. (1988) *The New S Language*. Wadsworth & Brooks/Cole.

See Also

`colSums` for row and column sums.

Examples

```
## Pass a vector to sum, and it will add the elements together.
sum(1:5)
```

```
## Pass several numbers to sum, and it also adds the elements.
sum(1, 2, 3, 4, 5)
```

```
## In fact, you can pass vectors into several arguments, and everything gets added.
sum(1:2, 3:5)
```

```
## If there are missing values, the sum is unknown, i.e., also missing, ....
sum(1:5, NA)
## ... unless we exclude missing values explicitly:
sum(1:5, NA, na.rm = TRUE)
```

[Package *base* version 3.2.2]

Notice how the format is:

```
name_of_function(input1,input2,...)
```

So how do we create this ourselves? Here is the syntax for writing your own function:

```
name_of_function <- function(arg1,arg2,...){
  # Code that gets executed when function is called
}
```

Let's see some examples.

Example 1

In [2]:

```
# Simple function, no inputs!
hello <- function() {
  print('hello!')
}
```

In [3]:

```
hello()
```

```
[1] "hello!"
```

Example 2

In [4]:

```
helloyou <- function(name) {  
  print(paste('hello ',name))  
}
```

In [5]:

```
helloyou('Sammy')
```

```
[1] "hello  Sammy"
```

Example 3

In [6]:

```
add_num <- function(num1,num2) {  
  print(num1+num2)  
}
```

In [7]:

```
add_num(5,10)
```

```
[1] 15
```

Default values

Notice that so far we've had to define every single argument in the function when using it, but we can also have default values by using an equals sign, for example:

In [8]:

```
hello_someone <- function(name='Frankie') {  
  print(paste('Hello ',name))  
}
```

In [9]:

```
# uses default  
hello_someone()
```

```
[1] "Hello  Frankie"
```

In [10]:

```
# overwrite default  
hello_someone('Sammy')
```

```
[1] "Hello  Sammy"
```

You'll see lots of built-in functions use default values for a variety of tasks, where the users will usually need a particular value.

Returning Values

So far we've only been printing out results, but what if we wanted to **return** the results so that we could assign them to a variable, we can use the *return* keyword for this task in the following manner:

In [12]:

```
formal <- function(name='Sam',title='Sir'){  
  return(paste(title, ' ',name))  
}
```

In [13]:

```
formal()
```

Out[13]:

'Sir Sam'

In [14]:

```
formal('Issac Newton')
```

Out[14]:

'Sir Issac Newton'

Notice how we aren't printing, we are returning, meaning we can assign this to a variable:

In [15]:

```
var <- formal('Marie Curie','Ms.')
```

In [16]:

```
var
```

Out[16]:

'Ms. Marie Curie'

This is the sort of syntax you want to use for your functions when you want to pass arguments to them, and then get some sort of result in return.

Scope

Scope is the term we use to describe how objects and variable get defined within R. When discussing scope with functions, as a general rule we can say that if a variable is defined only inside a function than its scope is limited to that function. For example, consider the following function:

In [24]:

```
# Multiplies input by 5  
times5 <- function(input) {  
  result <- input ^ 2  
  return(result)  
}
```

In [27]:

```
pow_two(4)  
result # Not defined outside the scope of the function  
input # Not defined outside the scope of the function
```

Out[27]:

16

Error in eval(expr, envir, enclos): object 'result' not found

Error in eval(expr, envir, enclos): object 'input' not found

These error indicate that these variables are only defined inside the *scope* of the function. So variables defined inside of a function are only defined (or redefined) inside of that function. However, variables assigned outside of the function are *global* variables, and the function will have access to them due to their scope. For example:

In [35]:

```
v <- "I'm global v"
stuff <- "I'm global stuff"

fun <- function(stuff){
  print(v)
  stuff <- 'Reassign stuff inside func'
  print(stuff)
}
```

In [36]:

```
print(v) #print v
print(stuff) #print stuff
fun(stuff) # pass stuff to function
# reassignment only happens in scope of function
print(stuff)
```

```
[1] "I'm global v"
[1] "I'm global stuff"
[1] "I'm global v"
[1] "Reassign stuff inside func"
[1] "I'm global stuff"
```

So what is happening above? The following happens

print(v) will check for the global variable v, the outer scope

print(stuff) will also check for the global variable stuff

fun(stuff) will accept an argument stuff, print out v, and then reassign stuff (in the scope of the function) and print out stuff. Notice two things:

- The reassignment of stuff only effects the scope of the stuff variable inside the function
- The fun function first checks to see if v is defined at the function scope, if not (which was the case) it will then search the global scope for a variable names v, leading to it printing out "I'm global v".

Check out the function below and make sure you understand it:

In [39]:

```
double <- function(a) {
  a <- 2*a
  a
}
var <- 5
double(var)
var
```

Out[39]:

10

Out[39]:

5

Great we've learned a lot! Now its time to put your understanding to the test with some questions!